



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

آزمایش دوم: طراحی مدار ترکیبی به کمک طرحواره

Experiment 2: Sequential Circuit Design Using Schematic Capture

اعضای گروه

کیارش شجاعی ارانی

سینا فعالیت

پارسا ملکیان

نام درس

آزمایشگاه طراحی سامانه‌های رقمی

نیم‌سال تابستان ۱۴۰۴-۱۴۰۵

گروه یکشنبه

نام استاد درس

دکتر اجلالی

فهرست مطالب

۱	مقدمه	۳
۱-۱	شرح مسئله	۳
۲	تعریف نشانه‌ها	۴
۳	شمارنده بالا/پایین	۴
۱-۳	فعال‌سازی و جهت شمارش هنگام ورود و خروج هم‌زمان	۵
۴	آشکارسازهای کران ظرفیت	۶
۵	کنترل در ورودی	۶
۶	طرح‌واره کامل مدار	۷
۷	محاسبه بسامد کاری	۸
۸	پیاده‌سازی با زبان توصیف سخت‌افزار	۸
۱-۸	پودمان شمارنده بالا/پایین	۹
۲-۸	پودمان اصلی مدار	۹
۹	شبیه‌سازی و اعتبارسنجی	۱۲
۱۰	طراحی در Quartus و ملاحظات پیاده‌سازی	۱۳
۱-۱۰	استخراج توصیف Verilog از طرح‌واره	۱۵
۱۱	نتیجه‌گیری	۱۷
آ	پیوست: میز آزمون کامل	۱۸
ب	پیوست: پروژه Quartus	۲۴

۱ مقدمه

هدف از این آزمایش، طراحی و پیاده‌سازی یک مدار ترتیبی^۱ با استفاده از امکانات طرح‌واره^۲ در نرم‌افزار Quartus است. مسئله مطرح‌شده، کنترل در ورودی و خروجی یک اتاق انتظار با ظرفیت محدود است که باید با کمک یک شمارنده بالا/پایین و چند مدار منطقی کمکی پیاده‌سازی شود.

عنوان رسمی این آزمایش در صورت مسئله «طراحی مدار ترکیبی به کمک طرح‌واره» آمده است، اما با توجه به این‌که خروجی‌های مدار (Open و Close) به حافظه (وضعیت شمارنده و وضعیت باز/بسته در) وابسته‌اند، مدار موردنیاز ذاتاً ترتیبی است؛ همین موضوع نیز در بیان هدف صورت مسئله («طراحی و پیاده‌سازی یک مدار ترتیبی») به صراحت آمده است. طراحی این گزارش بر همین مبنا انجام شده است.

۱-۱ شرح مسئله

اتاق انتظار یک اداره ظرفیت ۱۵ نفر دارد و دارای یک در ورودی و یک در خروجی است. کنار هر یک از این درها یک حسگر^۳ نصب شده که با عبور فرد، نشانه‌ای به مدت یک نبض ساعت^۴ تولید می‌کند: نشانه ورود با IN و نشانه خروج با OUT نشان داده می‌شود.

فردی که قصد ورود دارد باید دکمه Ent را فشار دهد. در ورودی تنها در صورتی باز می‌شود که در همان لحظه فشردن دکمه، تعداد افراد حاضر کمتر از ۱۵ نفر باشد و ساعت مجاز ورود (نشانه T) هنوز سپری نشده باشد. برای باز نگه‌داشتن در ورودی تا عبور کامل فرد، نشانه Open باید برابر ۱ نگه داشته شود.

در خروجی همواره باز است، مگر آن‌که تعداد افراد حاضر در اتاق به صفر برسد؛ در این حالت نشانه Close باید ۱ شود تا در خروجی بسته شود.

نکته مهم صورت مسئله این است که در یک نبض ساعت واحد ممکن است یک نفر از در ورودی وارد و هم‌زمان نفر دیگری از در خروجی خارج شود؛ همچنین حرکت در خلاف جهت تعیین‌شده برای هر در (مثلاً خروج از در ورودی) امکان‌پذیر نیست.

Sequential Logic^۱
Schematic^۲
Sensor^۳
Clock Pulse^۴

۲ تعریف نشانه‌ها

جدول ۱ نشانه‌های ورودی و خروجی مدار طراحی شده را نشان می‌دهد. همه نشانه‌ها نسبت به لبه بالارونده^۵ نشانه ساعت Clk همگام‌سازی می‌شوند.

جدول ۱: نشانه‌های ورودی و خروجی مدار

پهنا	جهت	شرح فارسی	Signal
۱ بیت	ورودی	نبض ساعت سامانه	Clk
۱ بیت	ورودی	بازنشانی سامانه	Rst
۱ بیت	ورودی	نبض حسگر در ورودی	IN
۱ بیت	ورودی	نبض حسگر در خروجی	OUT
۱ بیت	ورودی	دکمه درخواست ورود	Ent
۱ بیت	ورودی	مجاز بودن ساعت ورود	T
۱ بیت	خروجی	باز نگه داشتن در ورودی	Open
۱ بیت	خروجی	بستن در خروجی	Close
۴ بیت	داخلی/نمایشی	تعداد افراد حاضر در اتاق	Q (occupancy)

از آنجا که ظرفیت اتاق ۱۵ نفر است، تعداد افراد حاضر عددی بین صفر تا پانزده است و دقیقاً در بازه نمایش یک عدد چهاربیتی (از ۰۰۰۰ تا ۱۱۱۱) جای می‌گیرد؛ بنابراین شمارنده باید حداقل ۴ بیت باشد.

۳ شمارنده بالا/پایین

طبق راهنمای آزمایش، می‌توان از یک شمارنده بالا/پایین^۶ مطابق جدول ۲ استفاده کرد. ورودی Clr پاک‌سازی ناهمگام و فعال‌درپایین شمارنده است (مستقل از Clk به صفر بازمی‌گردد)، Enable فعال‌سازی همگام شمارش است و U جهت شمارش را تعیین می‌کند (۱ برای شمارش رو به بالا، ۰ برای شمارش رو به پایین).

^۵ Rising Edge
^۶ Up/Down Counter

جدول ۲: عملکرد شمارنده بالا/پایین

Function	Enable	Clr	Clk	U
Reset Counter to 0	X	0	X	X
Hold Previous Number	0	1	X	X
Up Count	1	1	↑	1
Down Count	1	1	↑	0

۱-۳ فعال سازی و جهت شمارش هنگام ورود و خروج همزمان

طبق صورت مسئله، ممکن است در یک نبض ساعت هم نبض IN و هم نبض OUT همزمان فعال شوند (یک نفر وارد و نفر دیگری خارج می شود). چون امکان حرکت در خلاف جهت هر دو وجود ندارد، این دو رویداد کاملاً مستقل از یکدیگرند و نیازی به منطق جهت یابی اضافه نیست. جدول ۳ تمام حالت های ممکن و مقدار لازم برای Enable و U شمارنده را نشان می دهد:

جدول ۳: تعیین Enable و U شمارنده بر اساس IN و OUT

توضیح	U	Enable	OUT	IN
هیچ عبوری رخ نداده - نگه داشتن عدد قبلی	X	۰	۰	۰
فقط خروج - شمارش رو به پایین	۰	۱	۱	۰
فقط ورود - شمارش رو به بالا	۱	۱	۰	۱
ورود و خروج همزمان - خنثی، نگه داشتن عدد قبلی	X	۰	۱	۱

از این جدول، معادلات زیر برای ورودی های شمارنده به دست می آید:

$$\text{Enable} = \text{IN} \oplus \text{OUT} \quad (1)$$

$$\text{U} = \text{IN} \quad (2)$$

بنابراین یک شمارنده بالا/پایین به تنهایی کافی است و نیازی به دو شمارنده یا منطق جمع/تفریق جداگانه نیست؛ چون بیشینه تغییر خالص تعداد افراد در یک نبض ساعت همواره در بازه $\{-1, 0, +1\}$ است.

۴ آشکارسازهای کران ظرفیت

دو شرط مرزی روی خروجی شمارنده ($Q_3Q_2Q_1Q_0$) بررسی می‌شود: کمتر از ۱۵ بودن (برای مجاز کردن ورود) و برابر صفر بودن (برای بستن در خروجی).

عدد ۱۵ برابر ۱۱۱۱ است؛ پس عدد فعلی کمتر از ۱۵ است اگر و تنها اگر همه بیت‌ها همزمان ۱ نباشند:

$$LT15 = \overline{Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0} \quad (۳)$$

یعنی یک دروازه NAND چهارورودی. به همین ترتیب، صفر بودن تعداد افراد یعنی همه بیت‌ها همزمان ۰ باشند:

$$Close = EQ0 = \overline{Q_3 + Q_2 + Q_1 + Q_0} \quad (۴)$$

که یک دروازه NOR چهارورودی است. چون Close فقط تابعی از وضعیت فعلی شمارنده است (و شمارنده خود در هر نبض ساعت به‌روز می‌شود)، نیازی به ثبات جداگانه برای Close نیست و می‌تواند به‌صورت ترکیبی از خروجی شمارنده گرفته شود.

۵ کنترل در ورودی

شرط باز شدن در ورودی باید دقیقاً «در لحظه فشردن دکمه» بررسی شود، نه در تمام مدتی که دکمه پایین نگه داشته می‌شود؛ در غیر این صورت، اگر کاربر دکمه را نگه دارد، نشانه Open حتی پس از عبور فرد نیز دوباره فعال می‌ماند. برای همین، ابتدا یک آشکارساز لبه روی Ent قرار می‌گیرد: مقدار Ent یک نبض ساعت در ثباتی به نام Ent_d ذخیره می‌شود و بر این اساس، Ent_pulse به شکل زیر ساخته می‌شود:

$$Ent_pulse = Ent \cdot \overline{Ent_d} \quad (۵)$$

شرط مجاز بودن ورود («کمتر از ۱۵ نفر» و «ساعت مجاز») با این نبض ترکیب می‌شود:

$$\text{Set} = \text{Ent_pulse} \cdot \text{LT15} \cdot T \quad (6)$$

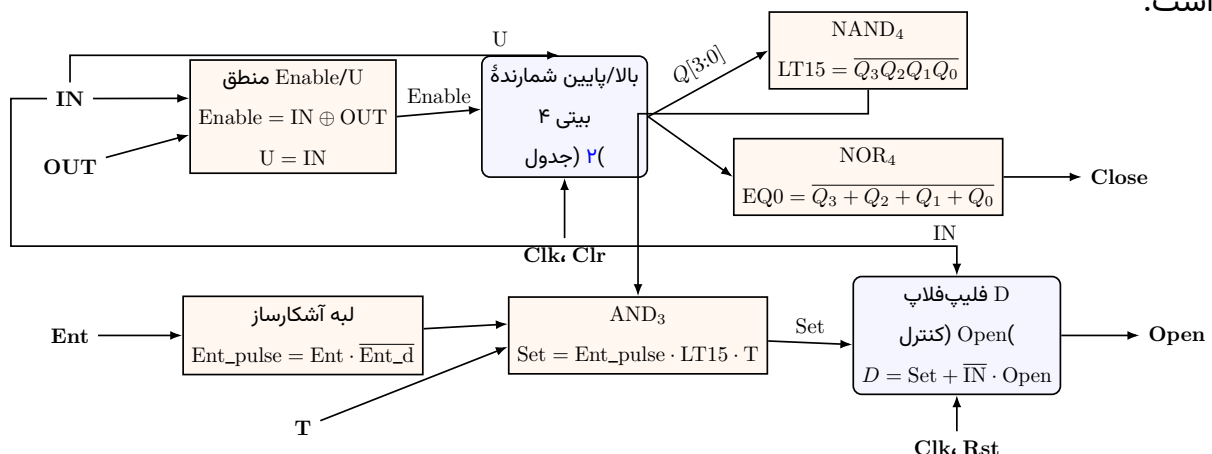
نشانه Open باید از لحظه برقراری Set تا لحظه عبور واقعی فرد (یعنی تا رسیدن نبض IN) در مقدار ۱ بماند؛ این دقیقاً رفتار یک لچ SR^v است که $\text{Set} = \text{Set}$ و $\text{Reset} = \text{IN}$. با استفاده از رابطه استاندارد تبدیل SR به D ($Q^+ = S + \bar{R}Q$)، ورودی D فلیپ‌فلاپ Open برابر است با:

$$D_{\text{Open}} = \text{Set} + \bar{\text{IN}} \cdot \text{Open} \quad (7)$$

یعنی به محض برقراری شرایط ورود، Open یک می‌شود و تا رسیدن نبض IN (عبور فرد) در همان مقدار می‌ماند، حتی اگر کاربر دکمه Ent را رها کرده یا دوباره فشار داده باشد؛ چون Set خود فقط برای یک نبض ساعت (به لطف آشکارساز لبه) فعال است.

۶ طرح‌واره کامل مدار

شکل ۱ طرح‌واره کامل مدار را نشان می‌دهد. بلوک شمارنده بالا/پایین (وسط شکل) دقیقاً مطابق جدول ۲ با چهار ورودی Clk، Clr، Enable، U و خروجی چهاربیتی Q رسم شده و ارتباط آن با بقیه مدار (منطق تعیین U/Enable در سمت چپ، و آشکارسازهای EQ0/LT15 در سمت راست) مشخص است.



شکل ۱: طرح‌واره کامل مدار کنترل اتاق انتظار

نشانه IN دو نقش دارد که هر دو در شکل مشخص شده‌اند: از یک سو مستقیماً ورودی U شمارنده را تعیین می‌کند، و از سوی دیگر مسیر بازخورد بازنشانی برای فلیپ‌فلاپ Open است (طبق رابطه ۷).

۷ محاسبه بسامد کاری

نبض‌های IN و OUT طبق صورت مسئله دقیقاً به اندازه یک نبض ساعت طول می‌کشند و بازتاب عبور فیزیکی یک فرد از در هستند. بنابراین بسامد^۸ کاری مدار نباید آن قدر بالا باشد که یک نبض ساعت کوتاه‌تر از زمان واقعی عبور یک نفر از در شود؛ در غیر این صورت حسگر نمی‌تواند نشانه‌ای معتبر و قابل تشخیص تولید کند. اگر فرض شود عبور کامل یک نفر از در دست کم حدود یک ثانیه زمان می‌برد (فرضی معقول برای عبور از یک در معمولی)، آنگاه:

$$T_{\text{clk}} \geq 1 \text{ s} \Rightarrow f_{\text{clk}} \leq 1 \text{ Hz} \quad (۸)$$

بنابراین بسامد کاری منطقی این مدار حدود $f_{\text{clk}} = 1 \text{ Hz}$ در نظر گرفته می‌شود. از سوی دیگر، برد FPGA معمولاً یک نوسان‌ساز مرجع با بسامد بسیار بالاتر (برای نمونه ۵۰ مگاهرتز) در اختیار می‌گذارد. برای پیاده‌سازی عملی روی برد، به جای ساخت یک ساعت مشتق‌شده^۹ ۱ هرتزی (که مشکلات رانش ساعت^۹ به همراه دارد)، بهتر است یک شمارنده تقسیم‌کننده بسامد جداگانه طراحی شود که هر ۵۰،۰۰۰،۰۰۰ نبض ساعت اصلی، یک نشانه فعال‌سازی^{۱۰} یک نبضی تولید کند؛ این نشانه به عنوان Enable اضافی به تمام ثبات‌های طراحی‌شده در بخش‌های پیشین متصل می‌شود تا مدار عملاً هر یک ثانیه یک بار به روزرسانی شود، بدون آن که شبکه ساعت اصلی برد دست‌کاری شود.

۸ پیاده‌سازی با زبان توصیف سخت‌افزار

برای اعتبارسنجی طراحی پیش از ورود به محیط طرح‌واره Quartus، مدار معادل با زبان Verilog نیز نوشته و شبیه‌سازی شد. هر بلوک طرح‌واره شکل ۱ دقیقاً به یک بخش از این کد (فهرست‌های ۱ و ۲) نگاشت می‌شود.

^۸Frequency
^۹Clock Skew
^{۱۰}Clock Enable

۱-۸ پودمان شمارنده بالا/پایین

```
1 // 4-bit up/down counter, function table matches the assignment's Table 1.
2 // Clr is active-low asynchronous clear; Enable is a synchronous count-enable;
3 // U selects count direction (1 = up, 0 = down) on the rising edge of Clk.
4 module updown_counter4 (
5     input wire    clk,
6     input wire    clr_n,
7     input wire    enable,
8     input wire    u,
9     output reg [3:0] q
10 );
11
12 always @(posedge clk or negedge clr_n) begin
13     if (!clr_n)
14         q <= 4'd0;
15     else if (enable) begin
16         if (u)
17             q <= q + 4'd1;
18         else
19             q <= q - 4'd1;
20     end
21 end
22
23 endmodule
```

Listing 1: Module: updown_counter4

۲-۸ پودمان اصلی مدار

```
1 // Waiting-room entrance/exit controller (Experiment 2).
2 //
```

```

3 // in_sensor / out_sensor : one-clock-pulse-wide sensor signals (IN / OUT)
4 // ent                    : entrance request push-button (Ent)
5 // t_allow                : 1 while entry hours are still open, 0 once the
6 //                        permitted entry hour has passed (T)
7 // rst                    : synchronous-style system reset button (active high)
8 // open_door              : held 1 from the moment entry is granted until
9 //                        the person is detected passing (IN) (Open)
10 // close_door             : 1 while the room is empty, so the exit door can
11 //                        be shut (Close)
12 // occupancy              : current headcount, 0..15 (debug / LED output)
13 module waiting_room (
14     input wire    clk,
15     input wire    rst,
16     input wire    in_sensor,
17     input wire    out_sensor,
18     input wire    ent,
19     input wire    t_allow,
20     output wire    open_door,
21     output wire    close_door,
22     output wire [3:0] occupancy
23 );
24
25 wire clr_n = ~rst;
26
27 // A person entering and a person leaving in the same clock pulse cancel
28 // out, so the counter only ever needs a single up/down step per cycle.
29 wire cnt_enable = in_sensor ^ out_sensor;
30 wire cnt_up     = in_sensor;
31
32 updown_counter4 U_COUNT (
33     .clk    (clk),
34     .clr_n  (clr_n),

```

```

35     .enable (cnt_enable),
36     .u      (cnt_up),
37     .q      (occupancy)
38 );
39
40 // occupancy < 15 <=> not all four bits are 1
41 wire lt15 = ~(occupancy[3] & occupancy[2] & occupancy[1] & occupancy[0]);
42 // occupancy == 0
43 wire eq0 = ~(occupancy[3] | occupancy[2] | occupancy[1] | occupancy[0]);
44
45 // Rising-edge detector on Ent, so the admission condition is evaluated
46 // exactly once "at the moment the button is pressed", not on every
47 // clock pulse the button happens to be held down.
48 reg ent_d;
49 always @(posedge clk or posedge rst)
50     if (rst) ent_d <= 1'b0;
51     else    ent_d <= ent;
52 wire ent_pulse = ent & ~ent_d;
53
54 wire set_open = ent_pulse & lt15 & t_allow;
55
56 reg open_reg;
57 always @(posedge clk or posedge rst)
58     if (rst) open_reg <= 1'b0;
59     else    open_reg <= set_open | (open_reg & ~in_sensor);
60
61 assign open_door = open_reg;
62 assign close_door = eq0;
63
64 endmodule

```

Listing 2: Module: waiting_room

۹ شبیه‌سازی و اعتبارسنجی

برای اعتبارسنجی منطق طراحی‌شده، یک میز آزمون^{۱۱} کامل با Icarus Verilog نوشته و اجرا شد (پیوست را ببینید). این میز آزمون یازده سناریو را پوشش می‌دهد که در جدول ۴ خلاصه شده‌اند.

جدول ۴: سناریوهای بررسی‌شده در میز آزمون

سناریو	نتیجه
وضعیت پس از بازنشانی (شمارنده صفر، ۰=Open، ۱=Close)	موفق
یک ورود ساده: فشردن Ent، باز شدن در تا رسیدن نبض IN	موفق
پر شدن اتاق تا ظرفیت (۱۵ نفر)	موفق
رد درخواست ورود شانزدهمین نفر (LT15 = 0)	موفق
یک خروج، بدون فعال شدن Close (اتاق هنوز پر نیست)	موفق
ورود مجدد پس از خالی شدن یک جا	موفق
ورود و خروج همزمان در یک نبض ساعت - بدون تغییر خالص	موفق
خالی کردن کامل اتاق و فعال شدن Close	موفق
رد ورود هنگامی که ۰=T (ساعت مجاز سپری شده)	موفق
باز شدن در پس از بازگشت T به ۱	موفق
نگه‌داشتن سطحی دکمه Ent: عدم فعال‌سازی مجدد Open در هر نبض	موفق
ساعت	

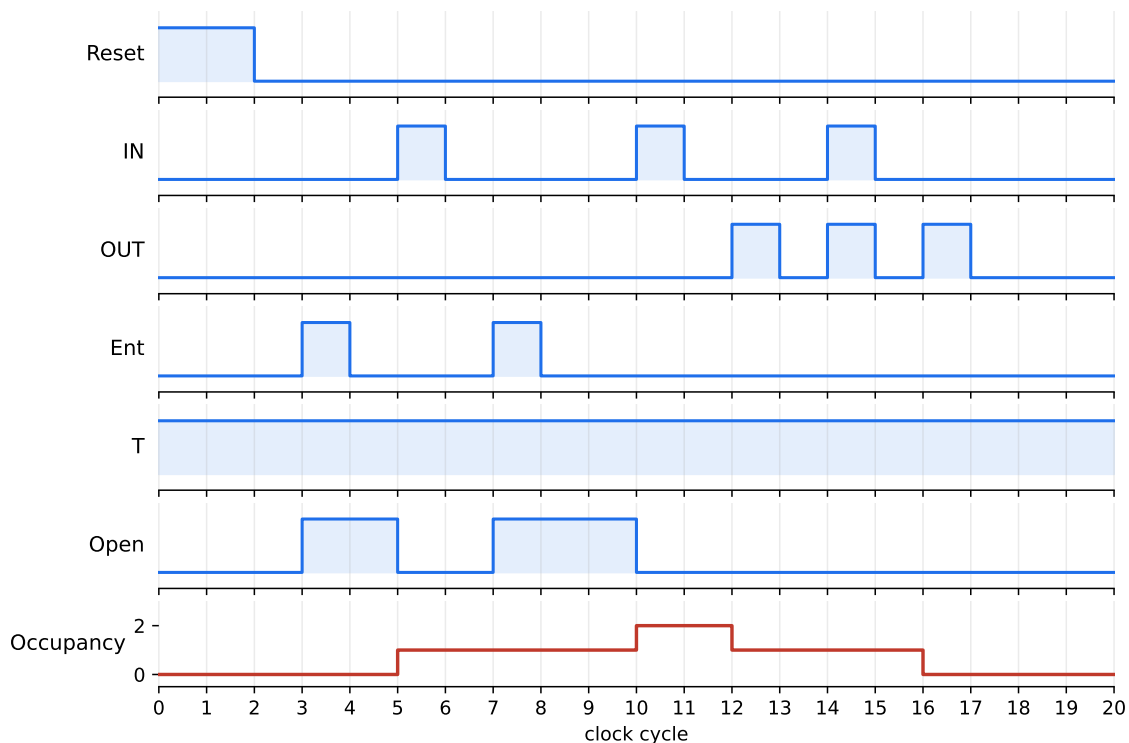
خروجی اجرای میز آزمون به صورت زیر است که موفقیت کامل تمام سناریوها را تأیید می‌کند:

ALL TESTS PASSED

شکل ۲ نمودار زمانی خروجی شبیه‌سازی را برای یک سناریوی کوتاه (دو ورود، یک خروج، یک ورود و خروج همزمان، و تخلیه کامل) نشان می‌دهد. مشاهده می‌شود که Open دقیقاً از لحظه پذیرش درخواست ورود تا رسیدن نبض IN فعال می‌ماند و مقدار Occupancy تنها همزمان با نبض‌های OUT/IN تغییر می‌کند.

شکل ۳ نیز نمای کامل موج‌نگار^{۱۲} همان اجرای یازده سناریویی میز آزمون را روی کل بازه زمانی شبیه‌سازی نشان می‌دهد؛ سیگنال errors در تمام طول اجرا برابر ۰ باقی می‌ماند، که موفقیت کامل همه سناریوها را تأیید می‌کند.

^{۱۱} Testbench - در متن صورت مسئله نیز همین واژه («فایل میز آزمون») به کار رفته است.
^{۱۲} Waveform Viewer

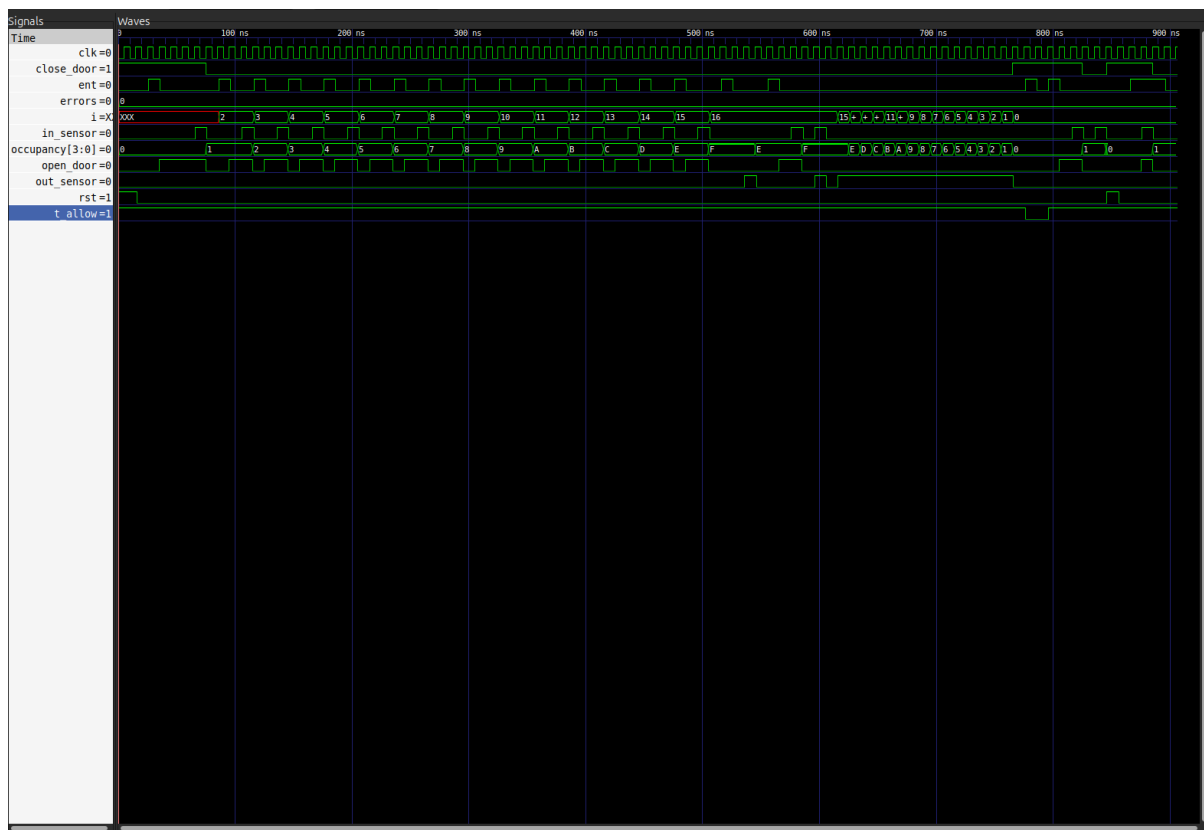


شکل ۲: نمودار زمانی شبیه‌سازی برای یک سناریوی نمونه

۱۰ طراحی در Quartus و ملاحظات پیاده‌سازی

طرح‌واره این مدار (فایل `waiting_room.bdf`) مستقیماً به فرمت طرح‌واره Quartus نوشته و در نرم‌افزار Quartus Prime Lite (نسخه ۱۰.۲۵، خانواده تراشه Cyclone IV E، تراشه EP4CE22F17C6) کامپایل شد. ساختار این طرح‌واره دقیقاً همان چیزی است که در شکل ۱ دیده می‌شود: بلوک شمارنده بالا/پایین به صورت یک زیرمدار (به ماژول `updown_counter4` ارجاع می‌دهد)، و بقیه مدار از نمادهای اولیه Quartus – `AND2`، `AND3`، `AND4`، `NAND4`، `NOR4`، `OR2`، `XOR` و فلیپ‌فلاپ DFF – ساخته شده است. طرح‌واره ابتدا از طریق ابزارهای خط فرمان خود Quartus (`quartus_map`، `quartus_fit`، `quartus_asm` و `quartus_sta`) نوشته و اعتبارسنجی شد، و سپس در محیط گرافیکی Quartus باز و بررسی شد. شکل ۴ نمای RTL Viewer همین طرح‌واره را نشان می‌دهد – دقیقاً همان ساختاری که در شکل ۱ رسم شده، این بار به صورت خودکار از روی فایل `.bdf`. توسط خود Quartus بازسازی شده است.

کامپایل کامل (سنتر، جای‌گذاری/مسیریابی، و تحلیل زمان‌بندی) با موفقیت به پایان رسید و فایل برنامه‌ریزی `waiting_room.sof` تولید شد. جدول ۵ خلاصه منابع مصرفی روی تراشه را نشان می‌دهد و شکل ۵ نمای Technology Map Viewer را پس از جای‌گذاری/مسیریابی نشان می‌دهد – مداری



شکل ۳: نمای کامل موج‌نگار برای اجرای کامل میز آزمون یازده سناریویی

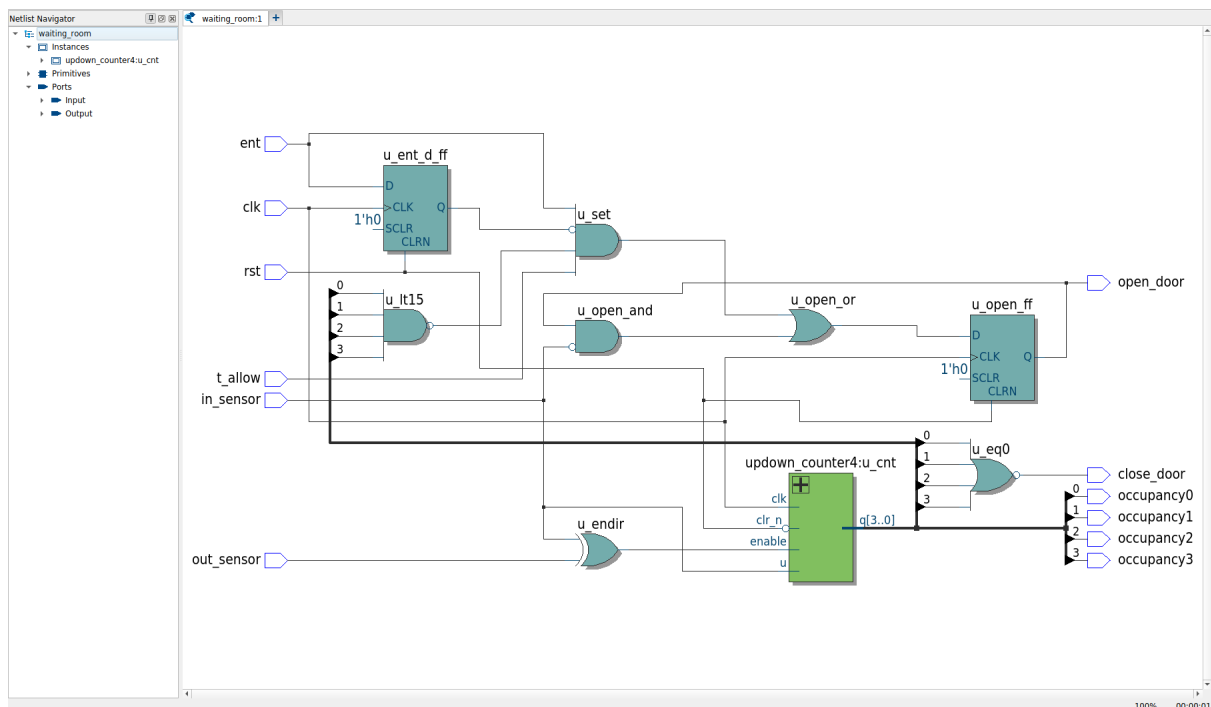
که واقعاً روی سلول‌های منطقی تراشه Cyclone IV E پیاده شده است.

جدول ۵: خلاصه منابع مصرفی پس از کامپایل کامل در Quartus

Resource	Usage
Total logic elements	10 / 22,320 (<1%)
Dedicated logic registers	6 / 22,320 (<1%)
Total pins	12 / 154 (8%)
Total memory bits	0 / 608,256 (0%)

برای اطمینان از تطابق کامل طرح‌واره با مدل Verilog ای که در فهرست ۲ شبیه‌سازی و اعتبارسنجی شد، همان کامپایل کامل یک‌بار دیگر مستقیماً از روی rtl/waiting_room.v نیز اجرا شد؛ هر دو مسیر - طرح‌واره گرافیکی و توصیف Verilog - دقیقاً به همان ۱۰ عنصر منطقی و ۶ ثبات جدول ۵ سنتز شدند، که هم‌ارزی کامل این دو توصیف از یک مدار واحد را تأیید می‌کند.

با تحلیل زمان‌بندی برای ساعت مرجع ۵۰ مگاهرتزی (طبق بخش ۷)، مدار به راحتی زمان‌بندی را برآورده می‌کند و بیشینه بسامد قابل دستیابی آن (F_{max}) در بدترین حالت دما/ولتاژ برابر با ۵۵۴



شکل ۴: نمای RTL Viewer در Quartus برای طرحواره waiting_room.bdf

مگاهرتز گزارش شد (که به دلیل محدودیت نرخ تعویض پایه‌های ورودی/خروجی این تراشه، تا ۲۵۰ مگاهرتز محدود می‌شود) – فاصله زیادی تا ساعت ۵۰ مگاهرتزی مرجع، که نشان می‌دهد مدار برای هیچ بسامد کاری معقولی محدودیت زمان‌بندی ندارد.

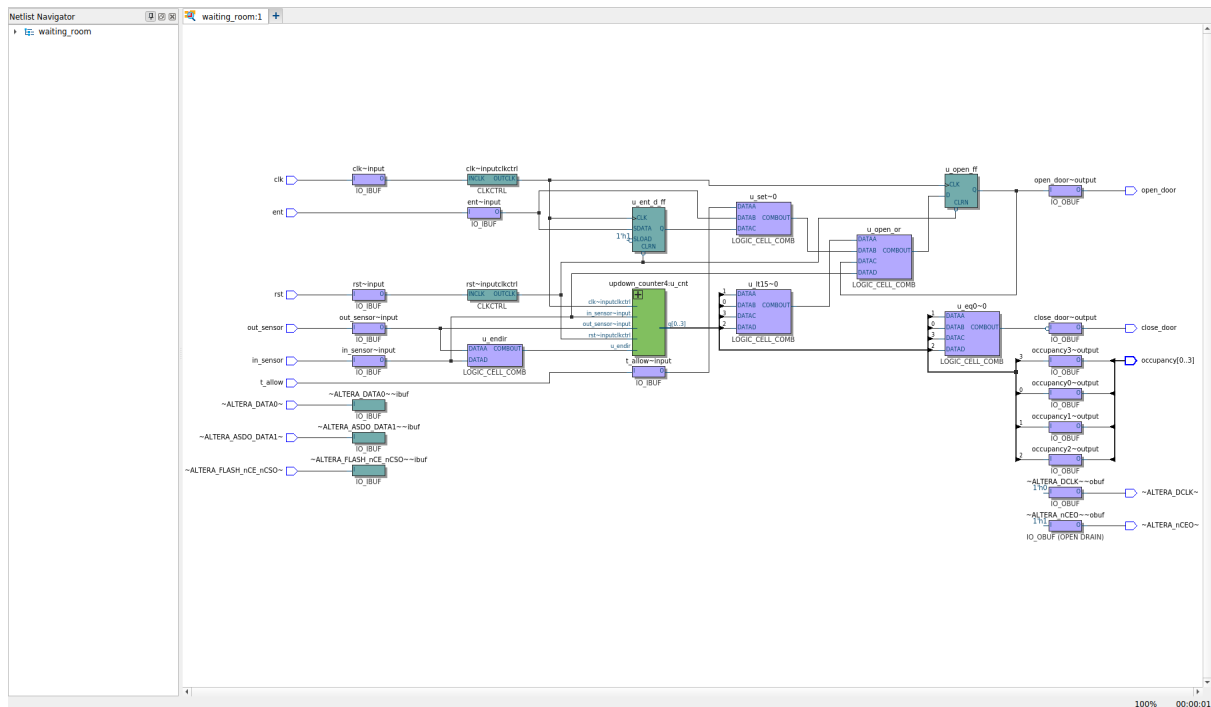
تخصیص پایه‌ها^{۱۳} در فایل .qsf. به صورت نمونه (برای خانواده تراشه Cyclone IV E) نوشته شده و باید پیش از پیاده‌سازی واقعی روی برد آزمایشگاه، متناسب با پایه‌های همان برد اصلاح شود.

۱-۱۰ استخراج توصیف Verilog از طرحواره

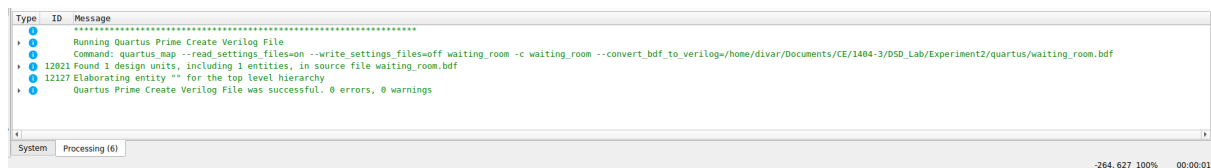
Quartus امکان تولید خودکار یک فایل Verilog معادل با هر طرحواره را از طریق منوی File → Create/Update → Create HDL Design File for Current File waiting_room.bdf از طریق محیط گرافیکی Quartus انجام شد؛ شکل ۶ خروجی پنجره Processing را نشان می‌دهد که موفقیت این عملیات را تأیید می‌کند.

فایل حاصل بررسی شد؛ اما بررسی دقیق‌تر نشان داد این فایل تولیدشده به‌طور خودکار دو اشکال واقعی دارد که هر دو از یک ریشه می‌آیند: ابزار تبدیل Quartus هنگام تخت‌کردن^{۱۴} سیم گذرگاهی

^{۱۳} Pin Assignment
^{۱۴} Flatten



شکل ۵: نمای (Post-Fitting) Technology Map در Quartus – مداري که واقعاً روی تراشه EP4CE22F17C6 پیاده‌سازی شده است



شکل ۶: خروجی پنجره Processing در Quartus پس از اجرای Create Verilog File روی waiting_room.bdf

چهاربیتی خروجی شمارنده به تک‌تک اتصالات یک‌بیتی که به ورودی‌های NAND4 و NOR4 می‌رفت، اطلاعات بیت خاص هر اتصال را گم کرده است:

– در چهار خط پایانی فایل، عبارت‌هایی مانند $assign\ occupancy0 = q[0]$ ؛ سیگنالی به نام q ارجاع می‌دهند که هرگز در فایل تعریف نشده است (نسخه درست این انتساب‌ها، با نام داخلی سیم گذرگاه، پیش‌تر در فایل آمده است؛ این چهار خط صرفاً تکراری و نامعتبرند).

– عبارت $close_door = \sim(SYNTHESIZED_WIRE_25 \mid SYNTHESIZED_WIRE_25)$ به جای OR کردن چهار بیت جداگانه گذرگاه، همان گذرگاه چهاربیتی را با خودش چهار بار OR کرده است – که از نظر منطقی برابر با خود گذرگاه است، نه NOR4 آن؛ پس از تخصیص این مقدار چهاربیتی به سیم

یک بیت `close_door`، فقط کم‌ارزش‌ترین بیت در نظر گرفته می‌شود که معادل با $\overline{Q_0}$ است، نه رابطه ۴. اشکال کاملاً مشابهی - AND کردن گذرگاه با خودش به جای چهار بیت جداگانه - برای سیگنال معادل LT15 (رابطه ۳) نیز رخ داده است.

این اشکال مربوط به مدار طراحی‌شده یا به کامپایل واقعی آن نیست - همان‌طور که در بالا دیده شد، کامپایل کامل طرح‌واره (که مستقیماً از روی `waiting_room.bdf` انجام می‌شود، نه از روی این فایل خروجی) به درستی به $Q_0 \cdot Q_1 \cdot Q_2 \cdot Q_3$ و $Q_0 + Q_1 + Q_2 + Q_3$ منتهی شده و دقیقاً همان ۱۰ عنصر منطقی و ۶ ثبات جدول ۵ را تولید کرده است. اشکال فقط در همین یک ابزار کمکی متن‌سازی (تبدیل طرح‌واره به فایل Verilog خوانا) وجود دارد.

هر دو نسخه - خروجی خام Quartus (`waiting_room_export_original.v`) و نسخه اصلاح‌شده (`waiting_room_export_fixed.v`) - همراه با گزارش ارسال شده‌اند. نسخه خام همان‌طور که هست حتی کامپایل هم نمی‌شود (به دلیل ارجاع به `q` تعریف نشده). برای اطمینان از درستی نسخه اصلاح‌شده، همان میز آزمون یازده سناریویی فهرست ۳ با تطبیق نام‌گذاری پایه‌های `occupancy0` تا `occupancy3` روی این فایل نیز اجرا شد (فایل `tb_export_check.v`) و نتیجه `ALL TESTS PASSED` را تکرار کرد - که هم‌ارزی کامل نسخه اصلاح‌شده با مدار طراحی‌شده را تأیید می‌کند.

۱۱ نتیجه‌گیری

مدار طراحی‌شده با یک شمارنده بالا/پایین چهاربیتی، دو آشکارساز کران ظرفیت ($NAND_4$ و NOR_4)، یک آشکارساز لبه برای دکمه `Ent`، و یک فلیپ‌فلاپ `D` برای کنترل `Open`، تمام شرایط صورت‌مسئله را پوشش می‌دهد: مدیریت ظرفیت ۱۵ نفره، رعایت ساعت مجاز ورود، بازماندن در ورودی تا عبور واقعی فرد، بستن در خروجی هنگام خالی شدن اتاق، و پردازش درست ورود و خروج هم‌زمان در یک نبض ساعت. صحت منطق طراحی‌شده با یک میز آزمون یازده سناریویی در Icarus Verilog تأیید شد، و طرح‌واره در Quartus Prime Lite با موفقیت کامپایل شد؛ کامپایل مستقل توصیف Verilog به همان تعداد عنصر منطقی و ثبات رسید، که هم‌ارزی دو توصیف را تأیید می‌کند.

آ پیوست: میز آزمون کامل

میز آزمون کامل که یازده سناریوی جدول ۴ را پوشش می‌دهد، در فهرست ۳ آمده است. فایل این کد (tb_waiting_room.v) همراه با گزارش ارسال شده است.

```
1 `timescale 1ns/1ps
2
3 module tb_waiting_room;
4
5     reg clk = 0;
6     reg rst;
7     reg in_sensor, out_sensor, ent, t_allow;
8     wire open_door, close_door;
9     wire [3:0] occupancy;
10
11     integer errors = 0;
12
13     waiting_room DUT (
14         .clk      (clk),
15         .rst      (rst),
16         .in_sensor (in_sensor),
17         .out_sensor (out_sensor),
18         .ent      (ent),
19         .t_allow   (t_allow),
20         .open_door (open_door),
21         .close_door (close_door),
22         .occupancy (occupancy)
23     );
24
25     always #5 clk = ~clk;
26
27     task tick;
28         begin
```

```

29         @(posedge clk);
30         #1;
31     end
32 endtask
33
34 task check_occ(input [3:0] expected, input [127:0] label);
35     begin
36         if (occupancy !== expected) begin
37             $display("FAIL [%0s] time=%0t occupancy=%0d expected=%0d", label,
38                 $time, occupancy, expected);
39             errors = errors + 1;
40         end
41     end
42 endtask
43
44 task check_bit(input got, input expected, input [127:0] label);
45     begin
46         if (got !== expected) begin
47             $display("FAIL [%0s] time=%0t got=%0b expected=%0b", label, $time
48                 , got, expected);
49             errors = errors + 1;
50         end
51     end
52 endtask
53
54 task press_ent(input allow, input under_cap);
55     begin
56         t_allow = allow;
57         ent = 1'b1;
58         tick;
59         ent = 1'b0;
60         tick;

```

```

59     end
60 endtask
61
62 task pass_in;
63     begin
64         in_sensor = 1'b1;
65         tick;
66         in_sensor = 1'b0;
67     end
68 endtask
69
70 task pass_out;
71     begin
72         out_sensor = 1'b1;
73         tick;
74         out_sensor = 1'b0;
75     end
76 endtask
77
78 integer i;
79
80 initial begin
81     $dumpfile("waiting_room.vcd");
82     $dumpvars(0, tb_waiting_room);
83     rst = 1; ent = 0; in_sensor = 0; out_sensor = 0; t_allow = 1;
84     tick; tick;
85     rst = 0;
86     tick;
87
88     // 1) reset state
89     check_occ(4'd0, "reset->occupancy0");
90     check_bit(close_door, 1'b1, "reset->close_door");

```

```

91     check_bit(open_door, 1'b0, "reset->open_door");
92
93     // 2) simple entrance: press Ent, door opens, stays open until IN pulse
94     press_ent(1, 1);
95     check_bit(open_door, 1'b1, "after ent->open");
96     tick; tick; // hold: still open, nobody passed yet
97     check_bit(open_door, 1'b1, "still waiting->open held");
98     pass_in;
99     tick;
100    check_bit(open_door, 1'b0, "after in pulse->open closes");
101    check_occ(4'd1, "after 1 entry->occupancy1");
102    check_bit(close_door, 1'b0, "occupancy1->close_door low");
103
104    // 3) fill the room to capacity (15)
105    for (i = 2; i <= 15; i = i + 1) begin
106        press_ent(1, 1);
107        pass_in;
108    end
109    tick;
110    check_occ(4'd15, "filled to capacity->occupancy15");
111
112    // 4) 16th person should be refused: Ent pressed at count==15 -> no
113    open
114    press_ent(1, 1);
115    check_bit(open_door, 1'b0, "room full->ent refused");
116
117    // stray IN pulse should NOT be generated by a real system when the
118    // door never opened; counter itself would still track a physical
119    // pulse if one occurred, but here we simply confirm the door logic
120    // blocked the request.
121
122    // 5) someone exits: occupancy 15 -> 14, exit door stays open (no close

```

```

    )
122     pass_out;
123     tick;
124     check_occ(4'd14, "after 1 exit->occupancy14");
125     check_bit(close_door, 1'b0, "occupancy14->close_door low");
126
127     // 6) now under capacity again, Ent should be able to open the door
128     press_ent(1, 1);
129     check_bit(open_door, 1'b1, "under capacity again->ent opens");
130     pass_in;
131     tick;
132     check_occ(4'd15, "back to 15->occupancy15");
133
134     // 7) simultaneous IN and OUT in the same clock pulse -> no net change
135     in_sensor = 1'b1; out_sensor = 1'b1;
136     tick;
137     in_sensor = 1'b0; out_sensor = 1'b0;
138     tick;
139     check_occ(4'd15, "simultaneous in&out->occupancy unchanged");
140
141     // 8) drain the room completely
142     for (i = 15; i >= 1; i = i - 1) begin
143         pass_out;
144     end
145     tick;
146     check_occ(4'd0, "drained->occupancy0");
147     check_bit(close_door, 1'b1, "occupancy0->close_door high");
148
149     // 9) T=0 (permitted hour passed) blocks entrance even with room empty
150     press_ent(0, 1);
151     check_bit(open_door, 1'b0, "t_allow=0->ent refused");
152

```

```

153 // 10) T=1 again works
154 press_ent(1, 1);
155 check_bit(open_door, 1'b1, "t_allow=1->ent opens again");
156 pass_in;
157 tick;
158 check_occ(4'd1, "final sanity->occupancy1");
159
160 // 11) holding Ent down (level, not a fresh pulse) must not re-open
161 // the door or re-trigger admission on every clock tick.
162 pass_in; // clears open_door from step 10's remaining state if any
163 rst = 1; tick; rst = 0; tick;
164 ent = 1'b1;
165 tick; // rising edge -> should open (occupancy 0 < 15, t_allow=1)
166 check_bit(open_door, 1'b1, "level ent->opens once on edge");
167 pass_in; // person passes, occupancy->1, open should clear even though
168 // ent still held
169 tick;
170 check_bit(open_door, 1'b0, "ent held down->open still clears on IN");
171 ent = 1'b0;
172 tick;
173
174 if (errors == 0)
175     $display("ALL TESTS PASSED");
176 else
177     $display("%0d TEST(S) FAILED", errors);
178
179 $finish;
180 end
181 endmodule

```

Listing 3: Testbench: tb_waiting_room

ب پیوست: پروژه Quartus

فایل تنظیمات پروژه Quartus (waiting_room.qsf) در فهرست ۴ آمده است. طرحواره اصلی مدار (waiting_room.bdf) نیز همراه با گزارش ارسال شده است؛ چون این فایل در قالب متنی داخلی Quartus برای هر گره و هر سیم طرحواره مختصات و توضیحات ترسیمی جداگانه ذخیره می‌کند، برای خوانایی در این گزارش نیامده و باید مستقیماً در Quartus باز شود.

```
1 # ----- #
2 #
3 # Quartus Prime project settings for Experiment 2 -- Waiting-room controller
4 #
5 # Top-level entity is the schematic (waiting_room.bdf), matching the
6 # assignment's requirement to use Quartus's schematic-capture facilities.
7 # It hierarchically instantiates updown_counter4.v (the 4-bit up/down
8 # counter block from Table 1) and wires it to discrete logic-gate/flip-flop
9 # primitives for the rest of the circuit -- exactly the structure shown in
10 # the report's block diagram. rtl/waiting_room.v is kept as a separate,
11 # independently-simulated RTL model (see sim/tb_waiting_room.v); it is not
12 # part of this project's compiled sources, to avoid a duplicate top-level
13 # entity name against the schematic.
14 #
15 # FAMILY/DEVICE below is a placeholder (Cyclone IV E, EP4CE22F17C6 -- a
16 # common DEO-Nano-class part). Change FAMILY/DEVICE and every pin assignment
17 # in the "Pin assignments" section to match the exact FPGA board used in the
18 # lab session before compiling on real hardware.
19 #
20 # ----- #
21
22 set_global_assignment -name FAMILY "Cyclone IV E"
23 set_global_assignment -name DEVICE EP4CE22F17C6
24 set_global_assignment -name TOP_LEVEL_ENTITY waiting_room
25 set_global_assignment -name ORIGINAL_QUARTUS_VERSION 25.1
```

```

26 set_global_assignment -name LAST_QUARTUS_VERSION "25.1std.0 Lite Edition"
27 set_global_assignment -name PROJECT_OUTPUT_DIRECTORY output_files
28 set_global_assignment -name MIN_CORE_JUNCTION_TEMP 0
29 set_global_assignment -name MAX_CORE_JUNCTION_TEMP 85
30 set_global_assignment -name ERROR_CHECK_FREQUENCY_DIVISOR 1
31 set_global_assignment -name SDC_FILE waiting_room.sdc
32
33 # ----- #
34 # Source files
35 # ----- #
36
37 set_global_assignment -name BDF_FILE waiting_room.bdf
38 set_global_assignment -name VERILOG_FILE ../rtl/updown_counter4.v
39
40 # ----- #
41 # Pin assignments (PLACEHOLDERS -- replace with the lab board's actual pins)
42 # ----- #
43
44 # set_location_assignment PIN_R8 -to clk
45 # set_location_assignment PIN_J15 -to rst
46 # set_location_assignment PIN_E1 -to in_sensor
47 # set_location_assignment PIN_E2 -to out_sensor
48 # set_location_assignment PIN_M1 -to ent
49 # set_location_assignment PIN_M2 -to t_allow
50 # set_location_assignment PIN_A4 -to open_door
51 # set_location_assignment PIN_A5 -to close_door
52 # set_location_assignment PIN_B4 -to occupancy0
53 # set_location_assignment PIN_B5 -to occupancy1
54 # set_location_assignment PIN_C4 -to occupancy2
55 # set_location_assignment PIN_C5 -to occupancy3
56
57 set_global_assignment -name PARTITION_NETLIST_TYPE SOURCE -section_id Top

```

```

58 set_global_assignment -name PARTITION_FITTER_PRESERVATION_LEVEL
    PLACEMENT_AND_ROUTING -section_id Top
59 set_global_assignment -name PARTITION_COLOR 16764057 -section_id Top
60 set_global_assignment -name ENABLE_OCT_DONE OFF
61 set_global_assignment -name ENABLE_CONFIGURATION_PINS OFF
62 set_global_assignment -name ENABLE_BOOT_SEL_PIN OFF
63 set_global_assignment -name USE_CONFIGURATION_DEVICE OFF
64 set_global_assignment -name CRC_ERROR_OPEN_DRAIN OFF
65 set_global_assignment -name OUTPUT_IO_TIMING_NEAR_END_VMEAS "HALF VCCIO" -rise
66 set_global_assignment -name OUTPUT_IO_TIMING_NEAR_END_VMEAS "HALF VCCIO" -fall
67 set_global_assignment -name OUTPUT_IO_TIMING_FAR_END_VMEAS "HALF SIGNAL SWING"
    -rise
68 set_global_assignment -name OUTPUT_IO_TIMING_FAR_END_VMEAS "HALF SIGNAL SWING"
    -fall
69 set_instance_assignment -name PARTITION_HIERARCHY root_partition -to | -
    section_id Top

```

Listing 4: waiting_room.qsf

پ پیوست: توصیف Verilog استخراج شده از طرحواره

همان‌طور که در بخش «استخراج توصیف Verilog از طرحواره» توضیح داده شد، Quartus به‌طور خودکار یک فایل Verilog معادل با طرحواره تولید می‌کند که در آن، اتصال تک‌تک بیت‌های گذرگاه Q به ورودی‌های NOR4/NAND4 به‌اشتباه به خود گذرگاه تبدیل شده بود. خطوط اصلی تفاوت میان خروجی خام و نسخه اصلاح شده به شرح زیر است:

```

1 --- waiting_room_export_original.v (raw Quartus output)
2 +++ waiting_room_export_fixed.v
3 @@ close_door / LT15-equivalent signal @@
4 - close_door = ~(SYNTHESIZED_WIRE_25 | SYNTHESIZED_WIRE_25 |
    SYNTHESIZED_WIRE_25 | SYNTHESIZED_WIRE_25);

```

```

5 + close_door = ~(SYNTHESIZED_WIRE_25[0] | SYNTHESIZED_WIRE_25[1] |
    SYNTHESIZED_WIRE_25[2] | SYNTHESIZED_WIRE_25[3]);
6
7 - SYNTHESIZED_WIRE_20 = ~(SYNTHESIZED_WIRE_25 & SYNTHESIZED_WIRE_25 &
    SYNTHESIZED_WIRE_25 & SYNTHESIZED_WIRE_25);
8 + SYNTHESIZED_WIRE_20 = ~(SYNTHESIZED_WIRE_25[0] & SYNTHESIZED_WIRE_25[1] &
    SYNTHESIZED_WIRE_25[2] & SYNTHESIZED_WIRE_25[3]);
9
10 @@ trailing duplicate/invalid lines (removed) @@
11 - assign occupancy0 = q[0]; // "q" is never declared -- leftover from
12 - assign occupancy1 = q[1]; // the exporter, duplicates the correct
13 - assign occupancy2 = q[2]; // assigns already given earlier in the
14 - assign occupancy3 = q[3]; // file against SYNTHESIZED_WIRE_25[i]

```

هر دو فایل کامل (waiting_room_export_fixed.v و waiting_room_export_original.v) همراه با گزارش ارسال شده‌اند. تنها نسخهٔ اصلاح‌شده – که با میز آزمون tb_export_check.v دوباره اعتبارسنجی شد – در ادامه به‌طور کامل آمده است.

```

1 // Copyright (C) 2025 Altera Corporation. All rights reserved.
2 // Your use of Altera Corporation's design tools, logic functions
3 // and other software and tools, and any partner logic
4 // functions, and any output files from any of the foregoing
5 // (including device programming or simulation files), and any
6 // associated documentation or information are expressly subject
7 // to the terms and conditions of the Altera Program License
8 // Subscription Agreement, the Altera Quartus Prime License Agreement,
9 // the Altera IP License Agreement, or other applicable license
10 // agreement, including, without limitation, that your use is for
11 // the sole purpose of programming logic devices manufactured by
12 // Altera and sold by Altera or its authorized distributors. Please
13 // refer to the Altera Software License Subscription Agreements
14 // on the Quartus Prime software download page.
15

```

```

16 // PROGRAM   "Quartus Prime"
17 // VERSION   "Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition"
18 // CREATED   "Sat Aug 8 23:16:09 2026"
19
20 module waiting_room(
21     clk,
22     rst,
23     in_sensor,
24     out_sensor,
25     ent,
26     t_allow,
27     close_door,
28     open_door,
29     occupancy0,
30     occupancy1,
31     occupancy2,
32     occupancy3
33 );
34
35
36 input wire clk;
37 input wire rst;
38 input wire in_sensor;
39 input wire out_sensor;
40 input wire ent;
41 input wire t_allow;
42 output wire close_door;
43 output wire open_door;
44 output wire occupancy0;
45 output wire occupancy1;
46 output wire occupancy2;
47 output wire occupancy3;

```

```

48
49 wire SYNTHESIZED_WIRE_24;
50 wire SYNTHESIZED_WIRE_1;
51 wire SYNTHESIZED_WIRE_3;
52 wire SYNTHESIZED_WIRE_4;
53 wire [3:0] SYNTHESIZED_WIRE_25;
54 wire SYNTHESIZED_WIRE_13;
55 wire SYNTHESIZED_WIRE_14;
56 wire SYNTHESIZED_WIRE_16;
57 wire SYNTHESIZED_WIRE_17;
58 wire SYNTHESIZED_WIRE_18;
59 wire SYNTHESIZED_WIRE_19;
60 wire SYNTHESIZED_WIRE_20;
61
62 assign open_door = SYNTHESIZED_WIRE_13;
63 assign occupancy0 = SYNTHESIZED_WIRE_25[0];
64 assign occupancy1 = SYNTHESIZED_WIRE_25[1];
65 assign occupancy2 = SYNTHESIZED_WIRE_25[2];
66 assign occupancy3 = SYNTHESIZED_WIRE_25[3];
67
68
69
70
71 updown_counter4 b2v_u_cnt(
72     .clk(clk),
73     .clr_n(SYNTHESIZED_WIRE_24),
74     .enable(SYNTHESIZED_WIRE_1),
75     .u(in_sensor),
76     .q(SYNTHESIZED_WIRE_25));
77
78 assign SYNTHESIZED_WIRE_1 = in_sensor ^ out_sensor;
79

```

```

80
81 DFF b2v_u_ent_d_ff(
82     .CLRN(SYNTHESIZED_WIRE_24),
83     .D(ent),
84     .CLK(clk),
85     .Q(SYNTHESIZED_WIRE_3));
86
87 assign SYNTHESIZED_WIRE_4 = ~SYNTHESIZED_WIRE_3;
88
89 assign SYNTHESIZED_WIRE_19 = ent & SYNTHESIZED_WIRE_4;
90
91 assign close_door = ~(SYNTHESIZED_WIRE_25[0] | SYNTHESIZED_WIRE_25[1] |
    SYNTHESIZED_WIRE_25[2] | SYNTHESIZED_WIRE_25[3]);
92
93 assign SYNTHESIZED_WIRE_14 = ~in_sensor;
94
95 assign SYNTHESIZED_WIRE_20 = ~(SYNTHESIZED_WIRE_25[0] & SYNTHESIZED_WIRE_25[1]
    & SYNTHESIZED_WIRE_25[2] & SYNTHESIZED_WIRE_25[3]);
96
97 assign SYNTHESIZED_WIRE_18 = SYNTHESIZED_WIRE_13 & SYNTHESIZED_WIRE_14;
98
99
100 DFF b2v_u_open_ff(
101     .CLRN(SYNTHESIZED_WIRE_24),
102     .D(SYNTHESIZED_WIRE_16),
103     .CLK(clk),
104     .Q(SYNTHESIZED_WIRE_13));
105
106 assign SYNTHESIZED_WIRE_16 = SYNTHESIZED_WIRE_17 | SYNTHESIZED_WIRE_18;
107
108 assign SYNTHESIZED_WIRE_24 = ~rst;
109

```

```
110 assign SYNTHESIZED_WIRE_17 = SYNTHESIZED_WIRE_19 & SYNTHESIZED_WIRE_20 &  
    t_allow;  
111  
112 endmodule
```

Listing 5: waiting_room_export_fixed.v – corrected Verilog exported from the schematic